

Learning-Native Adaptive Software Framework (LNASF)

Technical Concept and Architecture Specification

Peyman Salimi
salimipeyman@gmail.com
Alternate: pmnslm6344@gmail.com

Version 0.1
August 28, 2026

Contents

Keywords	2
Abstract	2
1 Motivation	2
2 Core Idea	3
3 What LNASF Is and Is Not	3
4 Learning as a Software-Component Capability	3
5 Native Learning	3
6 Scratch Learning	4
7 Prediction Is Not Action	4
8 Confidence and Uncertainty	4
9 Deterministic Fallback	5
10 Levels of Adaptation	5
10.1 Parameter Adaptation	5
10.2 Strategy Adaptation	5
10.3 Structural Adaptation	5
11 Learning Modes	5
11.1 Passive	5
11.2 Advisory	5
11.3 Adaptive	5
11.4 Autonomous	5
12 Learning Scope	5
12.1 Local Learning	5
12.2 Shared Learning	6
12.3 Hybrid Learning	6

13 Feedback Loop	6
14 Model Drift and Learning Failure	6
15 Privacy	6
16 Frontend Applications	6
17 Backend Applications	7
18 Runtime and Infrastructure Applications	7
19 Independence from Programming Language	7
20 Independence from Learning Algorithm	7
21 Developer Model	7
22 Evaluation	7
23 Prior Art and Related Work	8
24 Position Relative to Prior Art	8
25 Research Hypothesis	8
26 Potential Software Family	8
27 Development Roadmap	9
28 Limitations and Open Questions	9
29 Design Principles	9
30 Scope of This Disclosure	9
31 Citation	10
32 References	10

Keywords

Machine Learning; Artificial Intelligence; Self-Adaptive Software; Adaptive Software; Online Learning; Runtime Adaptation; Learning-Enabled Software; Learning-Native Software Components; Adaptive Software Components; Software Libraries; Software Frameworks; Runtime Intelligence; Native Machine Learning; Scratch Machine Learning; Predictive Software; Adaptive Systems; Feedback-Driven Software.

Abstract

Software has traditionally been built around rules that are designed in advance. Developers decide how a library should cache data, how a framework should schedule work, how a runtime should handle failures, or how a user interface should load resources.

That approach is predictable and often effective. However, real-world software rarely operates under stable conditions. Users behave differently over time, workloads change, network conditions fluctuate, and the cost of different execution strategies can vary significantly.

This document proposes the **Learning-Native Adaptive Software Framework (LNASF)**, a general architectural framework for building software components that can learn from their own runtime experience and use that knowledge to adapt future behavior.

The framework is intended for reusable software components, including libraries, frameworks, runtimes, developer tools, frontend systems, backend systems, and infrastructure components.

The central idea is a continuous feedback loop:

Observe → Learn → Predict → Decide → Adapt → Measure → Learn

LNASF does not claim to introduce machine learning, self-adaptive software, runtime adaptation, online learning, or learning-enabled components as new concepts. These areas have extensive prior research and established techniques.

The contribution proposed here is a unified architectural model for treating **learning as an explicit, reusable capability of software components**, with particular emphasis on native learning within the host ecosystem, scratch implementations of learning mechanisms, explicit decision policies, safety constraints, deterministic fallbacks, and applicability across different programming languages and software layers.

1. Motivation

Most software libraries and frameworks behave according to rules that are determined before deployment.

Examples include:

```
timeout = 500 ms
retryCount = 3
cacheTTL = 5 min
prefetch = enabled
concurrency = 4
```

These values and strategies may work well in development and testing, but they are not necessarily optimal under every runtime condition.

Users change their behavior. Workloads change. Network conditions fluctuate. Failure patterns evolve. Resource availability varies. A policy that was reasonable yesterday may not be the best policy tomorrow.

LNASF asks a practical engineering question:

Can a reusable software component observe its runtime experience, learn from that experience, and use the learned knowledge to make better decisions later?

2. Core Idea

The central concept of LNASF is a **Learning-Native Adaptive Software Component (LNASC)**.

Such a component has a normal deterministic implementation, but also contains a learning capability that can use runtime observations to improve selected decisions.

A simplified representation is:

$$\textit{Component} = \textit{Deterministic Baseline} + \textit{Learning Layer} + \textit{Adaptive Policy}$$

The learning layer does not replace the original software logic. It provides an additional source of information that can influence selected runtime decisions.

The fundamental learning loop is:

$$O_t \rightarrow M_t \rightarrow \hat{Y}_{t+1} \rightarrow D_t \rightarrow A_t \rightarrow R_t \rightarrow M_{t+1}$$

The model may be updated using:

$$M_{t+1} = U(M_t, O_t, R_t)$$

3. What LNASF Is and Is Not

LNASF is an architectural proposal rather than a single machine-learning algorithm or product.

It is intended to provide a common way of reasoning about what a software component should observe, learn, predict, decide, constrain, and evaluate.

LNASF does not claim to invent machine learning, online learning, self-adaptive software, autonomic computing, MAPE-K, runtime adaptation, predictive prefetching, adaptive caching, adaptive scheduling, or learning-enabled components. These areas are treated as prior art and foundations.

4. Learning as a Software-Component Capability

LNASF treats learning as an explicit capability of a reusable software component.

<pre>Application -> Reusable Software Component -> Deterministic Behavior -> Learning Capability -> Runtime Experience -> Adaptation</pre>

The intention is not to turn every component into an autonomous system. The intention is to make adaptive capability a deliberate architectural choice.

5. Native Learning

One of the design principles of LNASF is **native learning**.

The learning layer should, as far as practical, operate within the language and runtime ecosystem of the host software.

<pre>TypeScript Library -> TypeScript Learning Engine C# Library -> C# Learning Engine Go Library -> Go Learning Engine Rust Library -> Rust Learning Engine Python Library -> Python Learning Engine</pre>
--

The baseline architecture should not require a different programming language, an external inference service, or a remote machine-learning runtime merely to provide adaptive behavior.

```
Host Software
  -> Native Learning Layer
    -> Prediction
      -> Decision
        -> Adaptation
```

Native learning is a design principle proposed by LNASF. It is not presented as a historical first.

6. Scratch Learning

A reference implementation should not require a specialized machine-learning framework to execute the essential learning mechanism.

Learning methods may be constructed from arithmetic, probability, statistics, linear algebra, numerical methods, optimization, and data structures.

For example:

$$P(j|i) = \frac{N(i, j)}{N(i)}$$

or an online parameter update such as:

$$w_{t+1} = w_t - \eta \nabla L(w_t)$$

The purpose is transparency and portability of the learning mechanism rather than avoidance of every standard-library function.

7. Prediction Is Not Action

A central LNASF principle is:

$$Prediction \neq Action$$

The decision process may be modeled as:

$$Decision = f(Prediction, Confidence, ExpectedBenefit, ExpectedCost, Constraints)$$

One possible utility formulation is:

$$U(a|x) = E[Benefit(a)|x] - E[Cost(a)|x]$$

with:

$$a^* = \arg \max_a E[U(a|x)]$$

subject to applicable constraints.

8. Confidence and Uncertainty

For example:

$$P(A|X) = 0.54$$

may not justify an adaptation, while:

$$P(A|X) = 0.96$$

may justify it, depending on the cost and risk.

An adaptive component may therefore use:

$$Confidence \geq \tau$$

before enabling a given adaptation.

9. Deterministic Fallback

A learning-enabled component must remain useful when learning is unavailable or unreliable.

```
Learned Policy
-> Validate
  -> Safe? ---- Yes -> Adaptive Action
    \---- No -> Deterministic Baseline
```

Conceptually:

$$\textit{Behavior} = \textit{Deterministic Baseline} + \textit{Optional Adaptive Intelligence}$$

10. Levels of Adaptation

10.1 Parameter Adaptation

The component preserves its algorithm or strategy but changes parameters such as timeout, retry delay, cache TTL, debounce interval, concurrency limit, or prefetch threshold.

10.2 Strategy Adaptation

The component selects between predefined strategies, for example aggressive, balanced, and conservative prefetching.

10.3 Structural Adaptation

The component changes an execution path, composition, or selected implementation. This is treated as an advanced capability.

11. Learning Modes

11.1 Passive

Observe and learn without changing behavior.

11.2 Advisory

Generate predictions or recommendations while leaving the final decision to the application or developer.

11.3 Adaptive

Automatically apply adaptations permitted by policy.

11.4 Autonomous

Continuously optimize behavior within explicit safety, resource, and policy constraints.

12. Learning Scope

12.1 Local Learning

The model belongs to a component, application, session, process, device, or user.

12.2 Shared Learning

Observations from multiple instances contribute to a shared model.

12.3 Hybrid Learning

A shared model provides a baseline while local models adapt to local conditions:

$$M = M_{global} + M_{local}$$

13. Feedback Loop

The observed result of an action becomes evidence for subsequent learning:

```
Observation
-> Learning
    -> Prediction
        -> Decision
            -> Action
                -> Outcome
                    -> Evaluation
                        -> Model Update
                            -> Observation
```

14. Model Drift and Learning Failure

A learning-enabled component should consider cold start, insufficient data, concept drift, distribution changes, prediction error, adaptation instability, model degradation, and unexpected outcomes.

15. Privacy

For client-side systems, local-first learning is an important option:

```
User Behavior
-> Local Observation
    -> Local Model
        -> Local Adaptation
```

Remote aggregation should be optional, explicit, and governed by privacy requirements.

16. Frontend Applications

Possible applications include predictive prefetching, adaptive loading, adaptive caching, request prioritization, adaptive debouncing, learnable state behavior, and resource scheduling.

For example:

$$P(\text{Page}_B | \text{Page}_A) = 0.87$$

A client can combine that estimate with confidence and network cost before deciding whether to prefetch Page B. Predictive prefetching itself is established prior art and is included here only as an example application.

17. Backend Applications

A backend component may estimate:

$$P(success|errorType, service)$$

and choose among retry, delayed retry, or fail-fast behavior. Other applications include adaptive timeout, adaptive retry, adaptive concurrency, workload-aware scheduling, cache optimization, and resource allocation.

18. Runtime and Infrastructure Applications

Runtime and infrastructure components may observe CPU, memory, I/O latency, request rate, queue depth, failure rate, and workload patterns. A general workload loop can be expressed as:

$$Workload_t \rightarrow Prediction_t \rightarrow SchedulingDecision_t$$

19. Independence from Programming Language

The architecture is intentionally language-independent and can be implemented in TypeScript, C#, Java, Go, Rust, Python, and other ecosystems. The architecture is shared while implementation details remain native to the host environment.

20. Independence from Learning Algorithm

Potential implementations include frequency models, conditional probability, moving averages, Bayesian updating, Markov models, online gradient methods, contextual bandits, and other suitable methods.

21. Developer Model

An LNASF-enabled component should make explicit what it observes, learns, predicts, and changes, what constraints limit adaptation, what happens under low confidence, what the deterministic fallback is, and how success is measured.

22. Evaluation

An adaptive component should be compared against a deterministic baseline. Potential metrics include:

- Latency
- CPU utilization
- Memory usage
- Network cost
- Error rate
- Prediction accuracy
- Adaptation overhead
- False-adaptation rate

A general formulation is:

$$NetUtility = Benefit - Cost - AdaptationOverhead$$

23. Prior Art and Related Work

The proposal is positioned within established bodies of research.

Kephart and Chess articulated the vision of autonomic computing and self-managing systems. Self-adaptive software subsequently developed extensive feedback-driven models, including MAPE-K. Machine learning has been widely studied as a mechanism for supporting self-adaptation. REX demonstrated online learning for runtime emergent software systems. ML-DEECo is especially relevant because it explores machine-learning-enabled components in a component-based self-organizing architecture.

These works establish that learning, runtime adaptation, feedback loops, and learning-enabled components are not new concepts.

24. Position Relative to Prior Art

LNASF does not claim to invent:

Machine Learning Self-Adaptive Software Autonomic Computing MAPE-K Online Learning Runtime Adaptation Predictive Prefetching Adaptive Caching Adaptive Scheduling Learning-Enabled Components
--

Instead, the proposed framework focuses on the following combination:

Reusable Software Component + Native Learning + Prediction
+ Explicit Decision Policy + Safety Constraints
+ Deterministic Baseline + Feedback

This is presented as a technical proposal requiring implementation and empirical validation.

25. Research Hypothesis

Reusable software components can benefit from treating learning as an explicit, native, and constrained architectural capability, allowing selected runtime decisions to improve through accumulated experience while preserving deterministic fallback behavior.

26. Potential Software Family

Learning-Native Software +-- Frontend +-- Predictive Prefetch +-- Adaptive Loading +-- Adaptive Cache +-- Learnable State +-- Backend +-- Adaptive Retry +-- Adaptive Timeout

+-- Adaptive Scheduling
+-- Adaptive Concurrency
+-- Runtime
+-- Workload Prediction
+-- Resource Allocation
+-- Execution Adaptation
+-- Infrastructure
+-- Failure Prediction
+-- Recovery Policies
+-- Resource Optimization

27. Development Roadmap

1. Concept: formalize terminology, boundaries, principles, and architecture.
2. Mathematical Learning Core: implement simple learning mechanisms from fundamental primitives.
3. Reference Implementation: build the first learning-native component in TypeScript, with React as an initial demonstration.
4. Evaluation: compare adaptive and deterministic behavior under realistic workloads.
5. Advanced Learning: explore Bayesian models, Markov models, online learning, contextual models, and bandit-based methods.
6. Additional Ecosystems: investigate C#, Go, Rust, Python, and Java implementations.
7. Framework and Runtime Applications: investigate larger libraries, frameworks, runtimes, and infrastructure systems.

28. Limitations and Open Questions

Open questions include cold start, model drift, adaptation stability, learning cost, reproducibility, safety guarantees, privacy, and cross-language consistency.

29. Design Principles

1. Learning is a capability.
2. Native execution is preferred.
3. Reference learning cores should be understandable from fundamental primitives.
4. Prediction is not action.
5. Adaptation is constrained.
6. Deterministic fallback is mandatory.
7. Feedback matters.
8. Learning should be measurable.

30. Scope of This Disclosure

This document records, as Version 0.1, the author's proposed terminology, architectural organization, mathematical framing, design principles, and implementation directions for LNASF.

It intentionally distinguishes established prior art from the specific framework formulation proposed here.

This document is a technical disclosure. It is not legal advice and does not by itself create exclusive rights in abstract ideas or guarantee patentability.

31. Citation

Salimi, Peyman. (2026). *Learning-Native Adaptive Software Framework (LNASF): Technical Concept and Architecture Specification*, Version 0.1.

32. References

1. Kephart, J. O., & Chess, D. M. (2003). The Vision of Autonomic Computing. *IEEE Computer*, 36(1), 41-50. DOI: 10.1109/MC.2003.1160055.
2. Wong, T., Wagner, M., & Treude, C. (2022). Self-adaptive systems: A systematic literature review across categories and domains. *Information and Software Technology*.
3. Gheibi, O., Weyns, D., & Quin, F. (2021). Applying Machine Learning in Self-Adaptive Systems: A Systematic Literature Review. *ACM Transactions on Autonomous and Adaptive Systems*. DOI: 10.1145/3469440.
4. Porter, B., Grieves, M., Rodrigues Filho, R., & Leslie, D. (2016). REX: A Development Platform and Online Learning Approach for Runtime Emergent Software Systems. *Proceedings of OSDI 2016*, 333-348.
5. Töpfer, M., Abdullah, M., Kruliš, M., Bureš, T., & Hnětynka, P. (2022). ML-DEECo: A Machine-Learning-Enabled Framework for Self-organizing Components. *ACSOS 2022*. DOI: 10.1109/ACSOSC56246.2022.00033.
6. Töpfer, M., Abdullah, M., Bureš, T., Hnětynka, P., & Kruliš, M. (2023). Machine-learning abstractions for component-based self-optimizing systems. *International Journal on Software Tools for Technology Transfer*, 25, 717-731. DOI: 10.1007/s10009-023-00726-x.